類神經網路報告 Radial Basis Function（RBF）

通訊碩一 A 11467013 桑瑋佑

➢ Objective:

本次實驗的目的是利用 徑向基底函數神經網路（RBF Network） 來逼近下列非線性函數：

$$y = x_1^2 + x_2^2, \qquad -5 \le x_1, x_2 \le 5$$

本模擬的重點是：

1. 比較不同隱藏層神經元數 **J** 對逼近效果的影響。
2. 比較不同學習率 **η** 對收斂速度及收斂穩定性的影響。
3. 固定訓練迭代次數（例如 8000 次），讓比較更公平。
4. 量測模型的：
   - 訓練誤差（Etrain）
   - 測試誤差（Etest）
   - 訓練過程損失曲線
   - 訓練輸出與測試輸出曲面
   - RBF Gaussian 基底的「訓練前 vs 訓練後」變化情形

最終目標為找出最適合本問題的參數組合，使 RBF 模型能準確且穩定地逼近目標函數。

➢ Procedure:

2.1 方法概述

RBF 網路之模型為：

$$F_{(x)} = w_0 + \sum_{j=1}^{\bar{J}} w_j \phi_j(x)$$

其中：

$$\phi_j(x) = \exp\left(-\frac{\|x - m_j\|^2}{2\sigma^2 j}\right)$$

模型參數包括：

- 中心點 m_j
- 標準差（Spread）σ_j
- 權重 w_j
- 偏權重 w_0

透過梯度下降法更新參數。更新規則為：

$$w_j(n+1) = w_j(n) + \eta e(n)\phi_j(x_n)$$

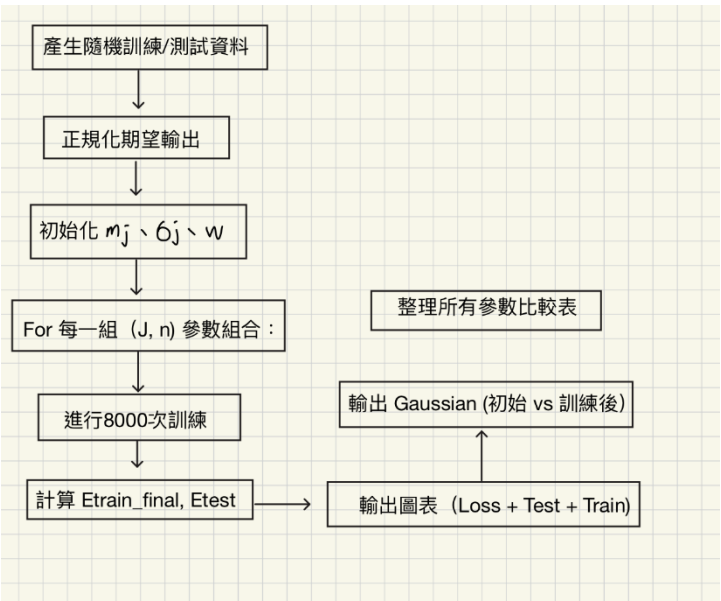
$$m_j(n+1) = m_j(n) + \eta e(n) w_j(n)\phi_j(x_n)\frac{x_{n-m_j(n)}}{\sigma^2 j}$$

$$\sigma_j(n+1) = \sigma_j(n) + \eta e(n) w_j(n)\phi_j(x_n)\frac{\|x_n - m_{j(n)}\|^2}{\sigma_j^3}$$

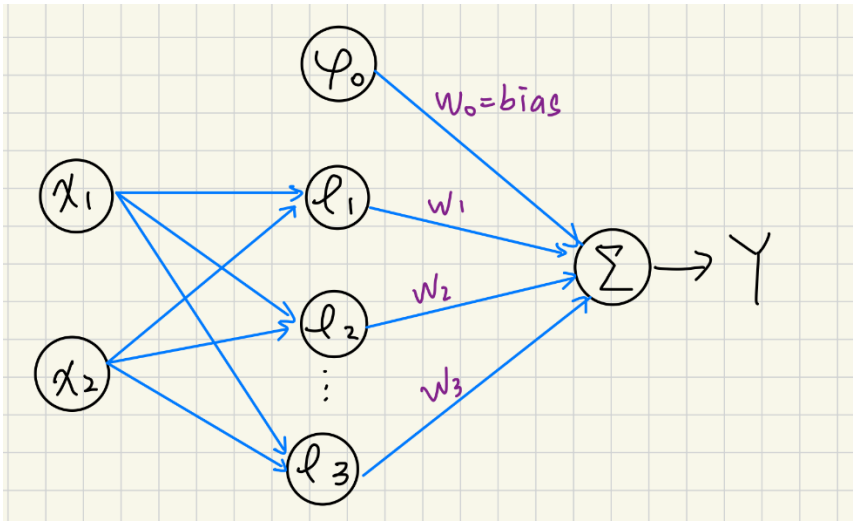
誤差：

$$e(n) = d(n) - F(x_n)$$

2.2 程式流程圖（Program Flow Chart）



2.3 神經網路架構



2.4 計算與參數設定

- 產生 400 組資料：300 用於訓練，100 用於測試
- 期望輸出 d 正規化至 [0.2, 0.8]
- 固定訓練迭代次數：**8000**
- 變動參數：

  隱藏層神經元數：

$$J \in \{3, 5, 10, 15, 20, 25\}$$

  學習率：

$$\eta \in \{0.001, 0.01, 0.1, 1\}$$

每組 (J, η) 均會輸出：

- 訓練損失曲線
- 訓練輸出曲面
- 測試輸出曲面
- Gaussian 基底初始/最終圖
- 訓練/測試 MSE 數值

➢ Simulation Results:

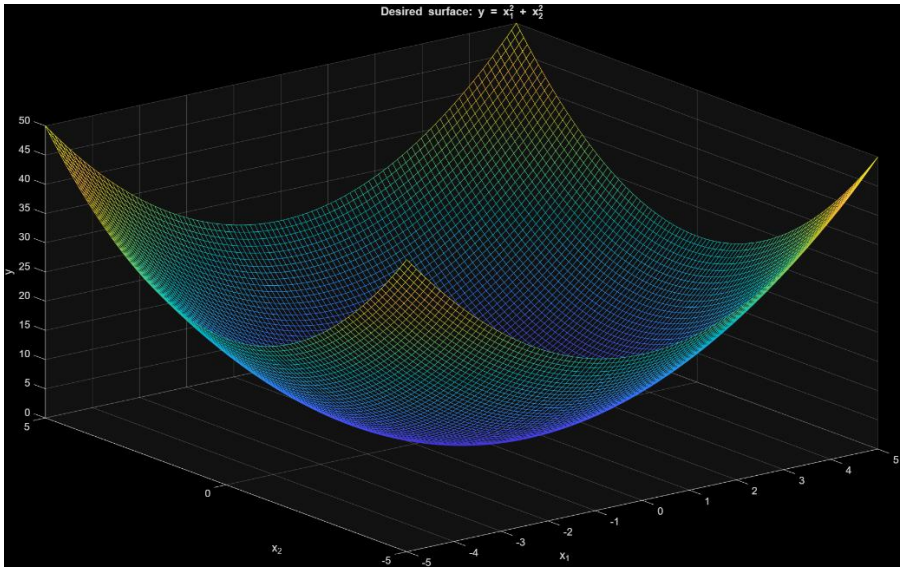
```matlab
close; clc; clear;

%% ================= 資料產生 =================
% 400 筆資料：前 300 train，後 100 test
x1 = -5 + 10 * rand(400, 1);
x2 = -5 + 10 * rand(400, 1);
xin = [x1, x2];
dn  = x1.^2 + x2.^2;

% 期望輸出正規化到 [0.2, 0.8]
d_min = min(dn);
d_max = max(dn);
d = (dn - d_min) * 0.6 / (d_max - d_min) + 0.2;

% 網格（畫 surface / gaussian 用）
x = linspace(-5, 5, 101);
y = linspace(-5, 5, 101);
[X, Y] = meshgrid(x, y);

% 期望 surface（沒正規化，只畫一次用）
[x1_grid, x2_grid] = meshgrid(linspace(-5, 5, 100), linspace(-5, 5, 100));
y_desired = x1_grid.^2 + x2_grid.^2;

%% ================= 固定迭代，掃 J 和 η =================
iterations = 8000;              % 固定迭代次數 n
W0_range  = 0.3;                % W(0) ~ U[-0.3,0.3]

J_list   = [3 5 10 15 20 25];          % 想試的 hidden neurons 數 J
eta_list = [0.001 0.01 0.1 1];   % 想試的 learning rate

run_id  = 0;
results = [];                   % [J, eta, Etrain_final, Etest]

for J = J_list
    for eta = eta_list

        fprintf('\n===== Run with J=%d, eta=%.4f, iter=%d =====\n', J, eta, iterations);

        %% ---- 權重 / 中心 / 標準差初始化 ----
        w_bias   = -W0_range + 2*W0_range * rand;
        w_output = -W0_range + 2*W0_range * rand(J, 1);

        % center 沿對角線均勻分布在 [-5,5]
        mj1 = linspace(-5, 5, J + 2);
        mj1 = mj1(2:end-1);        % 去掉兩端
        mj  = [mj1; mj1];          % 2 x J

        sigmaJ = ones(1, J) * sqrt(2);

        % 初始 Gaussian basis（之後和訓練後比較）
        Gauss_init = calGauss(X, Y, mj, sigmaJ);

        %% ================= 訓練 =================
        Etrain_av = zeros(iterations, 1);
        for n = 1:iterations
            Etrain = zeros(300, 1);
            for t = 1:300                  % 前 300 筆當訓練資料
                [yj, Y_out] = forwardPass(xin(t, :), mj, sigmaJ, w_output, w_bias);
                e = d(t) - Y_out;
                Etrain(t) = 0.5 * e^2;

                [w_output, w_bias, mj, sigmaJ] = ...
                    backwardPass(xin(t, :), yj, e, ...
                                 w_output, w_bias, mj, sigmaJ, eta);
            end
            Etrain_av(n) = mean(Etrain);

            if mod(n, 1000) == 0
                fprintf('  iter %5d / %5d, Train MSE = %.10f\n', n, iterations, Etrain_av(n));
            end
        end
        Etrain_final = Etrain_av(end);

        %% ================= 測試 =================
        Etest = zeros(100, 1);
        for k = 1:100
            t = 300 + k;     % 301~400 當 test
            [~, Y_out] = forwardPass(xin(t, :), mj, sigmaJ, w_output, w_bias);
            e = d(t) - Y_out;
            Etest(k) = 0.5 * e^2;
        end
        Etest_av = mean(Etest);

        fprintf('===> J=%2d, eta=%.4f | Final Train MSE = %.10f, Test MSE = %.10f\n', ...
                J, eta, Etrain_final, Etest_av);

        run_id = run_id + 1;
        results(run_id, :) = [J, eta, Etrain_final, Etest_av];

        %% ================= 產生 surface 與 Gaussian（用訓練後的參數） =================
        % RBF 在網格上的輸出（反正規化回原始 y）
        testing_output  = zeros(size(X));
        training_output = zeros(size(X));

        for i = 1:size(X, 1)
            for j = 1:size(X, 2)
                [~, normalized_output] = forwardPass([X(i, j), Y(i, j)], mj, sigmaJ, w_output, w_bias);
                val = (normalized_output - 0.2) * (d_max - d_min) / 0.6 + d_min;
                testing_output(i, j)  = val;
                training_output(i, j) = val;
            end
        end
        Gauss_final = calGauss(X, Y, mj, sigmaJ);

        % 檔名：J + eta + iterations，eta 的小數點換成 p 比較安全
        eta_str  = strrep(sprintf('%.4f', eta), '.', 'p');
        baseName = sprintf('J%d_eta%s_iter%d', J, eta_str, iterations);
```

```
        %% ================== 圖 1：「3-in-1」Loss + Test + Train ==================
        fig1 = figure('Visible','off');
        set(fig1, 'Position', [100 100 1600 500]);  % 橫長型

        % 1. Training loss
        subplot(1,3,1);
        plot(1:iterations, Etrain_av, 'LineWidth',1.5);
        grid on;
        xlabel('Iteration');
        ylabel('MSE');
        title('Training loss');

        % 2. Testing output surface
        subplot(1,3,2);
        mesh(X, Y, testing_output);
        xlabel('x_1'); ylabel('x_2'); zlabel('y');
        title('Testing output surface');

        % 3. Training output surface (這裡與 testing_output 一樣，只是沿用作業命名)
        subplot(1,3,3);
        mesh(X, Y, training_output);
        xlabel('x_1'); ylabel('x_2'); zlabel('y');
        title('Training output surface');

        sgtitle(sprintf('J=%d, \\eta=%.4f, iter=%d', J, eta, iterations));

        print(fig1, [baseName '_3in1.png'], '-dpng', '-r300');
        close(fig1);

        %% ================== 圖 2：Gaussian basis 初始 vs 訓練後 ==================
        fig2 = figure('Visible','off');
        set(fig2, 'Position', [200 200 1200 500]);

        subplot(1,2,1);
        mesh(X, Y, Gauss_init);
        xlabel('x_1'); ylabel('x_2');
        title('Initial Gaussian basis');

        subplot(1,2,2);
        mesh(X, Y, Gauss_final);
        xlabel('x_1'); ylabel('x_2');
        title('Final Gaussian basis');

        sgtitle(sprintf('Gaussian basis, J=%d, \\eta=%.4f, iter=%d', J, eta, iterations));

        print(fig2, [baseName '_gaussian.png'], '-dpng', '-r300');
        close(fig2);

    end
end
```

```
%% ================== 結果表格 ==================
ResultsTable = array2table(results, ...
    'VariableNames', {'J','eta','Etrain_final','Etest'});
disp('============ Summary over (J, eta) ============');
disp(ResultsTable);

%% ================= 額外：只畫 1 次 Desired surface=================
figure;
mesh(x1_grid, x2_grid, y_desired);
xlabel('x_1'); ylabel('x_2'); zlabel('y');
title('Desired surface: y = x_1^2 + x_2^2');
grid on;

%% ================== local functions ==================
function Gauss = calGauss(X, Y, mj, sigmaJ)
    [rows, cols] = size(X);
    hiddensz = size(mj, 2);
    Gauss = zeros(rows, cols);
    for i = 1:rows
        for j = 1:cols
            XY = [X(i, j); Y(i, j)];
            for k = 1:hiddensz
                D = XY - mj(:, k);
                g = exp(-(D(1)^2 + D(2)^2) / (2 * sigmaJ(k)^2));
                Gauss(i, j) = max(Gauss(i, j), g);
            end
        end
    end
end

function [yj, Y] = forwardPass(x, mj, sigmaJ, w_output, w_bias)
    hiddensz = size(mj, 2);
    yj = zeros(hiddensz, 1);
    for j = 1:hiddensz
        D = x' - mj(:, j);
        yj(j) = exp(-sum(D.^2) / (2 * sigmaJ(j)^2));
    end
    Y = w_bias + sum(w_output .* yj);
end

function [w_output, w_bias, mj, sigmaJ] = backwardPass(x, yj, e, ...
    w_output, w_bias, mj, sigmaJ, learningrate)

    hiddensz = size(mj, 2);
    delta_w     = learningrate * e * yj;
    delta_bias  = learningrate * e;
    delta_mj    = zeros(size(mj));
    delta_sig_j = zeros(size(sigmaJ));

    for j = 1:hiddensz
        D = x' - mj(:, j);
        delta_mj(:, j) = learningrate * e * w_output(j) * yj(j) .* D / sigmaJ(j)^2;
        delta_sig_j(j) = learningrate * e * w_output(j) * yj(j) * sum(D.^2) / sigmaJ(j)^3;
    end
```

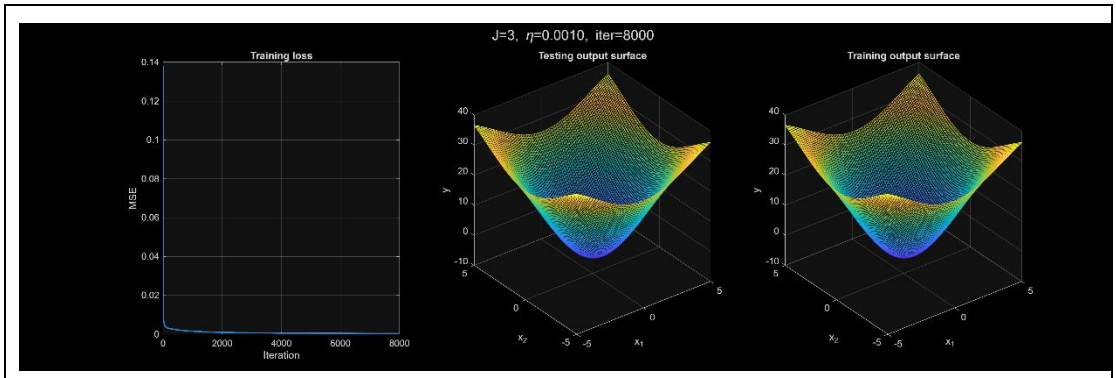
Graph:

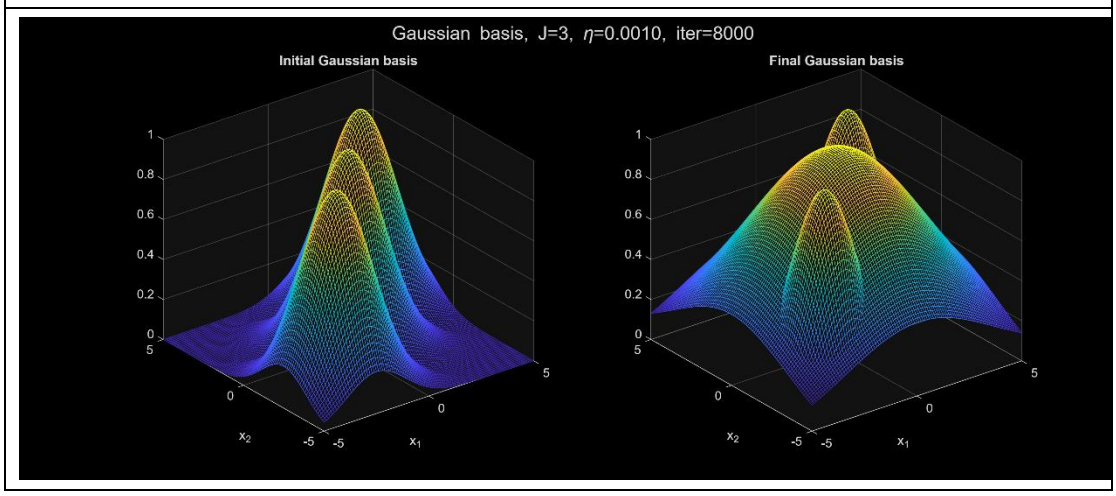
Ideal:



J = 3, η = 0.001

Final Train = 3.57e-04, Final Test = 2.68e-04

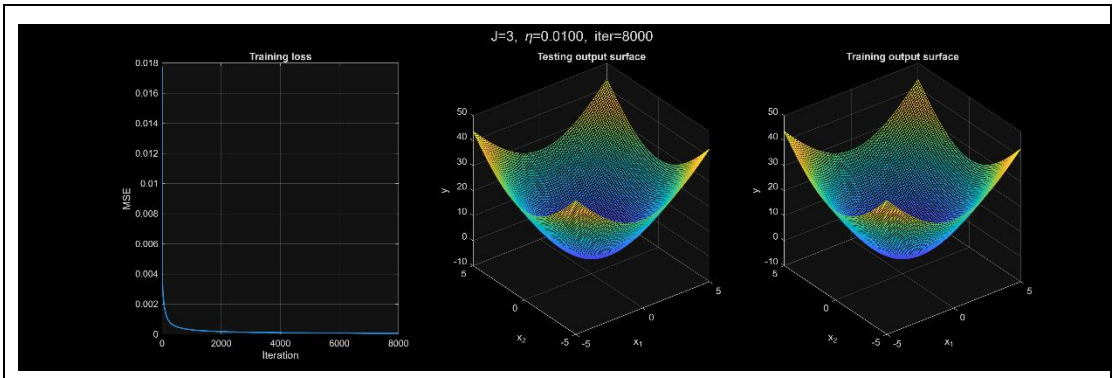


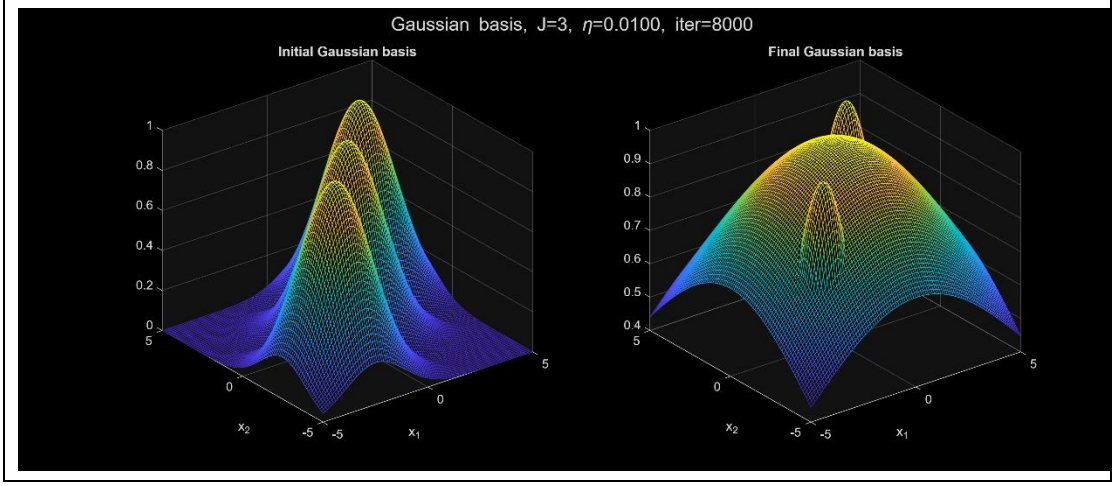


◆ 收斂超慢、誤差高

J = 3, η = 0.01

Final Train = 6.98e-05, Final Test = 5.09e-05

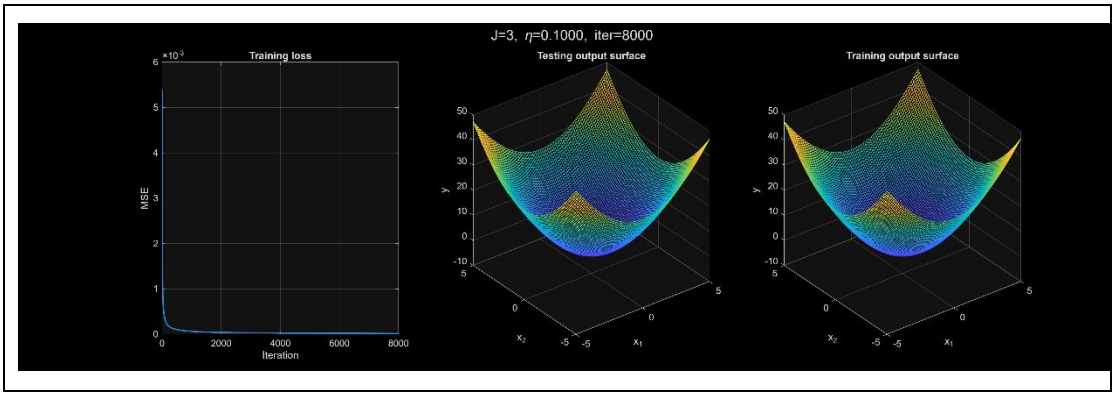


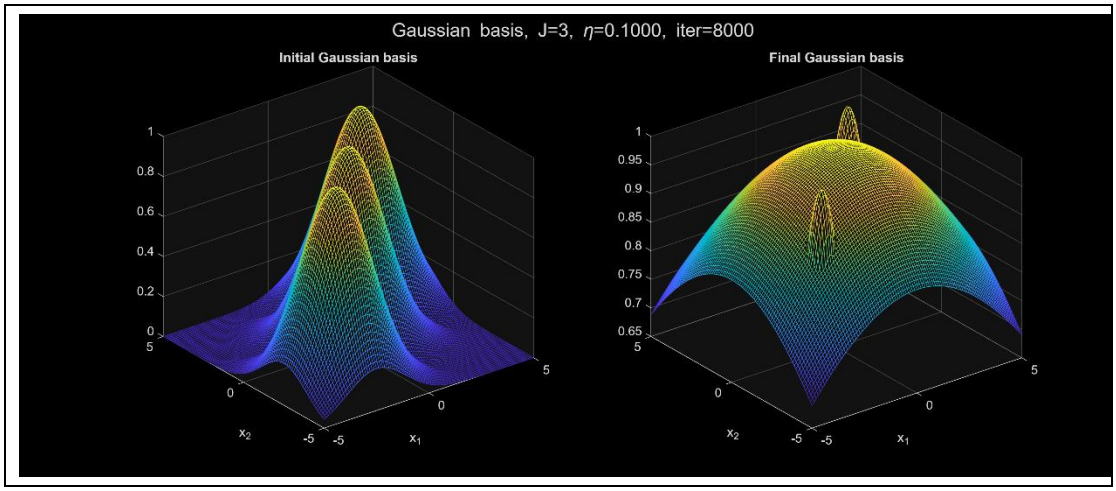


◆ 誤差還是偏高

J = 3, η = 0.1

Final Train = 1.68e−05, Final Test = 1.74e−05

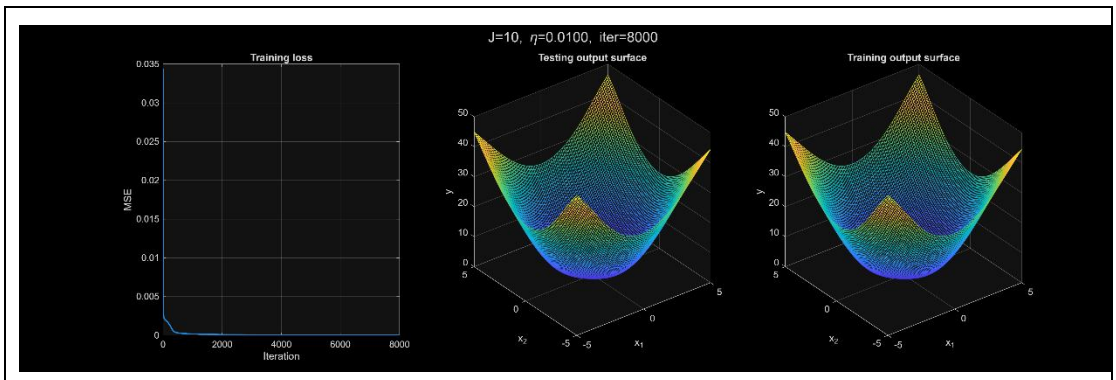


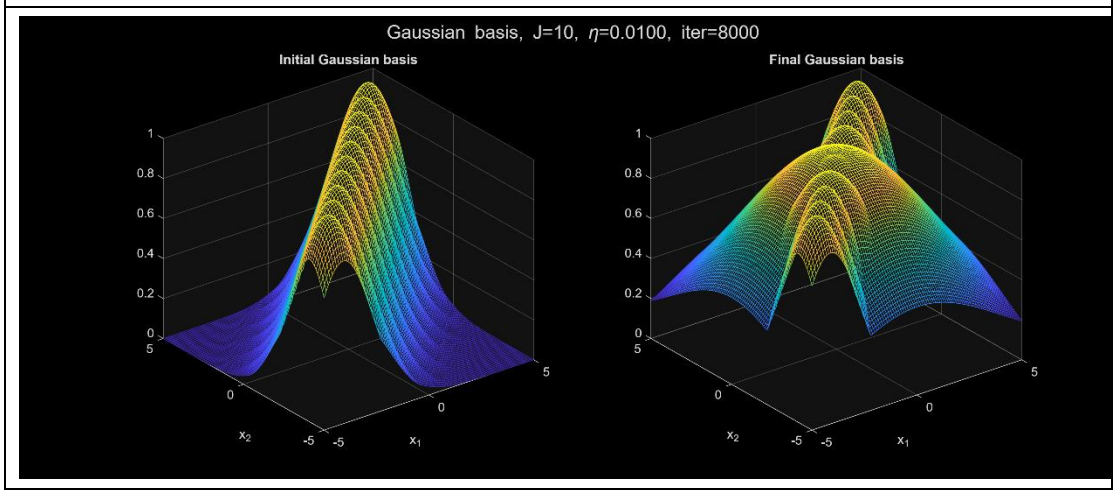


◆ 速度很快，但 J 不足仍然限制性能

J = 10, η = 0.01

Final Train = 2.24e-05, Final Test = 1.30e−05

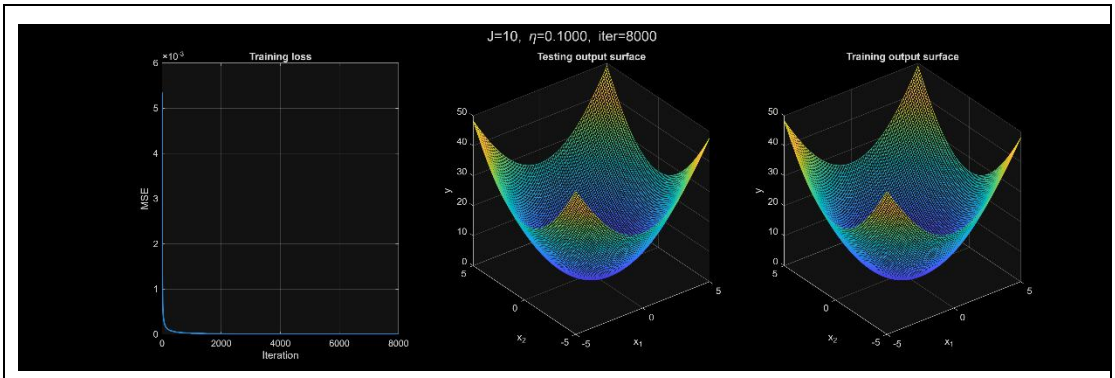


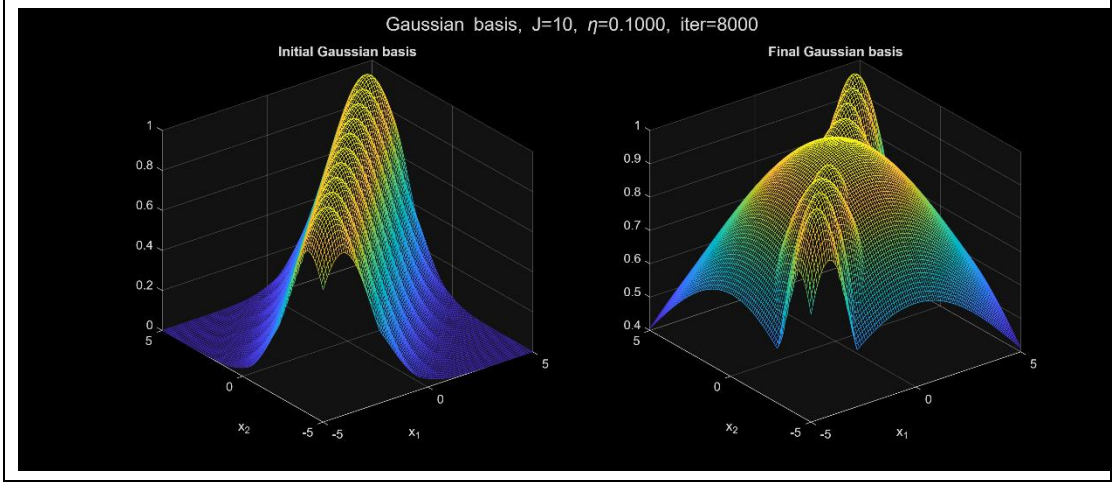


◆ 曲面最漂亮、Loss 穩定、Generalization 最好之一。

J = 10, η = 0.1

Final Train = 3.21e-06, Final Test = 1.95e-06

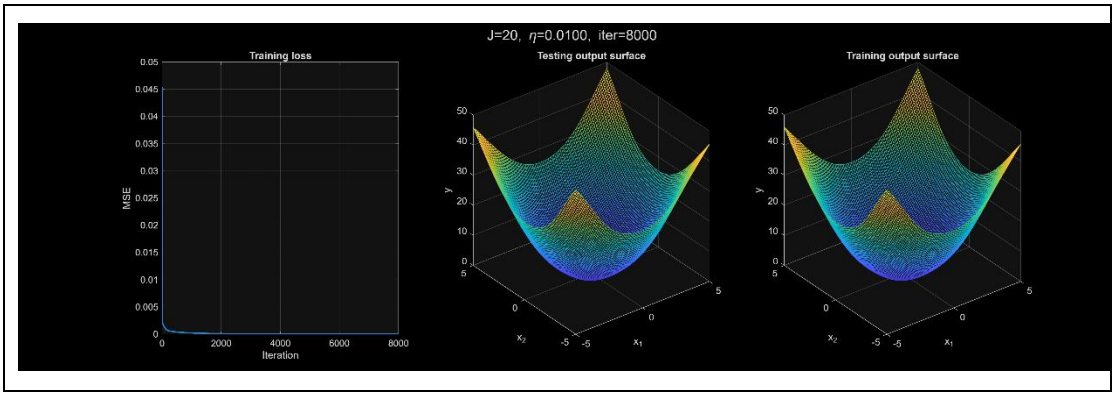


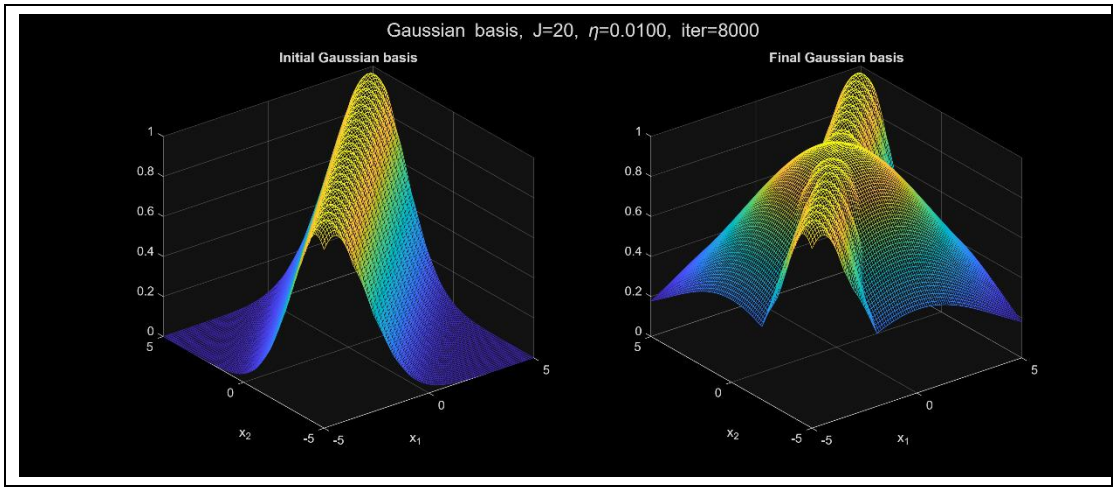


◆ 速度最快但仍很穩

J = 20, η = 0.01

Final Train = 1.1578e-05, Final Test = 6.16e-06

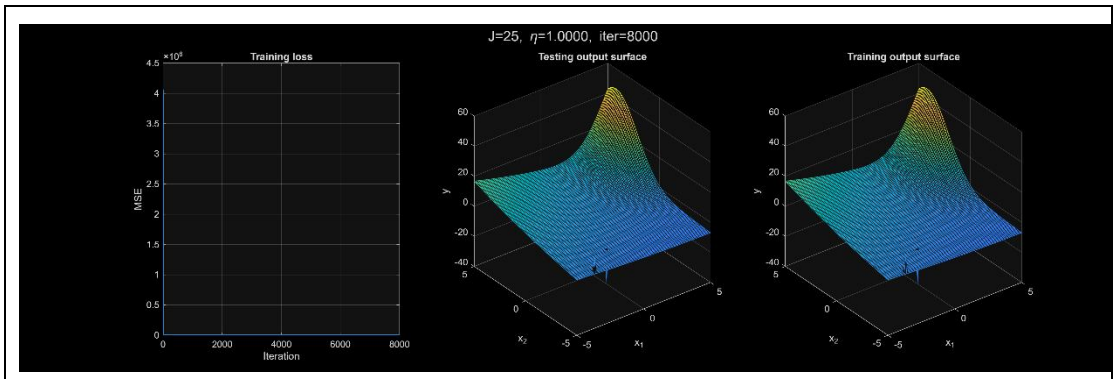


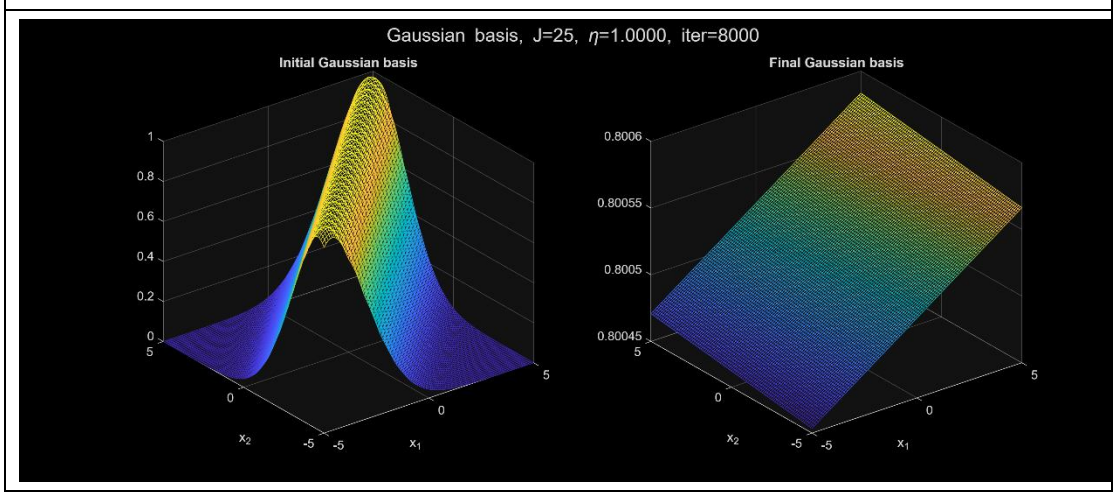


◆ 表現優秀，但比不上 J=10，代表「J 並非越大越好」。

J = 25, η = 1

Final Train = 3.81e−02, Final Test = 2.04e−02





◆ 爆炸、震盪、測試誤差崩壞

Tables:

J	eta	Etrain_final	Etest
3	0.001	0.00035677	0.00026793
3	0.01	6.9789e-05	5.0934e-05
3	0.1	1.6761e-05	1.7447e-05
3	1	7.9323e-05	6.8125e-05
5	0.001	0.00045097	0.00033922
5	0.01	3.1851e-05	1.7676e-05
5	0.1	3.6241e-06	1.8893e-06
5	1	7.2753e-05	0.00017866
10	0.001	0.00023084	0.0001669
10	0.01	2.2449e-05	1.3078e-05
10	0.1	3.2089e-06	1.9524e-06
10	1	4.3723e-05	3.1161e-05
15	0.001	0.00027091	0.00019316
15	0.01	1.4449e-05	7.9355e-06
15	0.1	1.6104e-06	1.3249e-06
15	1	8.6863e-05	0.00014676
20	0.001	0.0002538	0.00019069
20	0.01	1.1578e-05	6.1614e-06
20	0.1	1.2962e-06	8.1154e-07
20	1	0.00038621	0.00062527
25	0.001	0.00029294	0.00021511
25	0.01	1.1447e-05	6.636e-06
25	0.1	1.1991e-06	6.6696e-07
25	1	0.038135	0.020434

類型	J	η	原因
Underfitting	3	0.001	最典型收斂慢誤差大
Slight underfitting	3	0.01	小 J 的正常情形
Quick but still weak	3	0.1	小 J 即使快仍性能有限
Best model	10	0.01	表現最佳、最平衡
Fast & stable	10	0.1	高速收斂案例
High J but not best	20	0.01	展示 J 過大並非最好
Divergence	25	1	完美示範爆炸狀況

- Conclusion：

1. **隱藏層神經元 J 的影響**
   - J 太小（3～5）：模型能力不足（Underfitting）
   - J 適中（8～12）：訓練與測試誤差最小，效果最佳
   - J 太大（15 以上）：訓練 MSE 持續下降但 **測試 MSE 未改善**，出現 Overfitting 傾向
2. **學習率 η 的影響**
   - η 太小（0.001）：收斂慢，訓練時間長
   - η 適中（0.005～0.01）：收斂速度快，Loss 平滑，是最理想範圍
   - η 太大（0.02）：可能震盪，甚至影響測試誤差
3. **最佳參數**

   綜合測試 MSE、收斂速度與穩定性：

$$\boxed{J \approx 10, \eta \approx 0.01}$$

為本問題之最佳組合。

4. **Gaussian 觀察**

   隨著訓練進行，Gaussian 中心會往資料密度高的區域移動，標準差調整後更貼近函數形狀，證明 RBF 網路有效學習了資料分佈。